

CSE307

Principles of Programming Languages

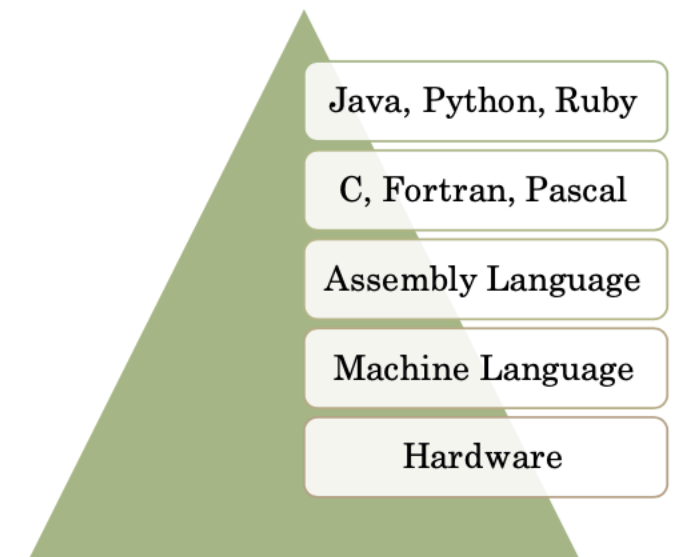
State University of New York, Korea

**Course
homepage**



Programming Languages Evolution

- Most programming languages are *high-level* languages, where the phrase “high-level” indicates a higher degree of abstraction.
- The second word of this course’s name – **abstraction** – is among the most important ideas in programming.
- It refers to the degree to which the language’s features are separated away from the details of a particular computer’s architecture and/or implementation at *lower* levels.



Abstraction

- A round, fluffy cat of 15 kilos with light orange and white fur
- A round, fluffy cat of 15 kilos
- A round cat
- Cat
- Animal



This course

- Syntax and grammar
- Lambda calculus
- Understanding programming languages via OCaml and a bit of C

Logistics

Meet the Instructor

- Man email: <zhoulai dot fu at sunykorea dot ac dot kr>
- Affiliated faculty with CS, SBU and ECE, VT
- CSE113 and CSE307
- Research Interest: Programming Language Theory, Software Security, AI systems; floating-point computation
- Previous Work: France, Spain, US, Denmark
- Education: École Polytechnique, France
- Personal: Happily married; no special hobbies

Course website

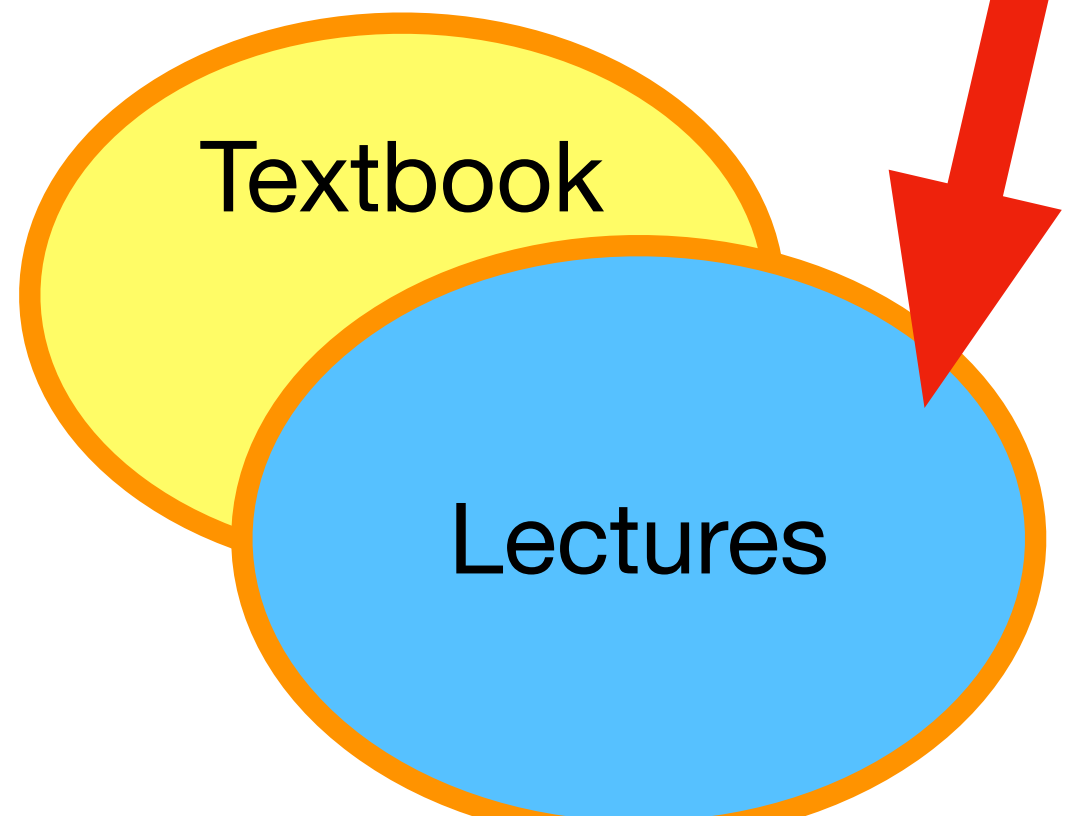
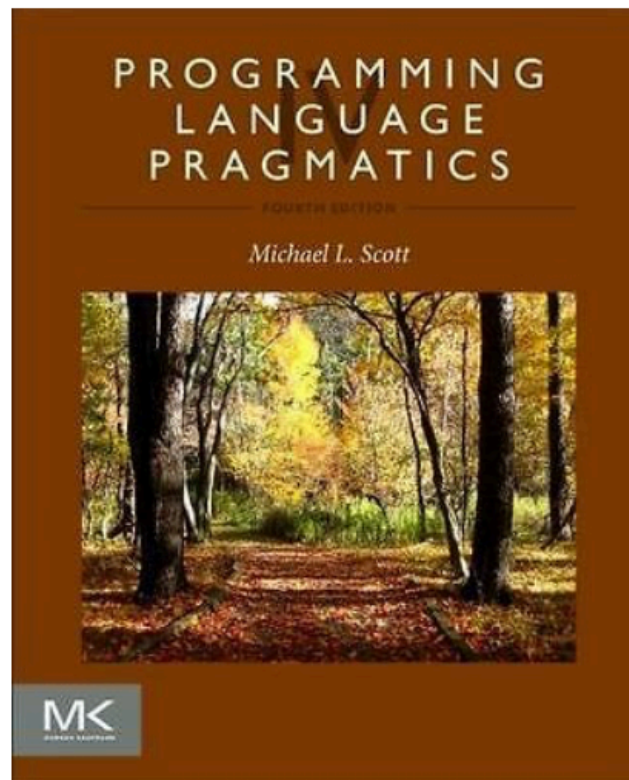
https://github.com/zhoulaiifu/26_cse307_fall



Reference books and reading material

- Michael L. Scott. Programming Language Pragmatics.
- For details pertaining to specific programming languages, the recommended material will mostly be from the following:
Python tutorial: <https://docs.python.org/3/tutorial/>
The official OCaml learning material from <https://ocaml.org/learn/>
- Other reading material (if used) will be added to the website for this course.

Exam



Questions so far?

Quiz

- Where to find official course info?
- Homework due time?
- How attendance will be checked?
- How late homework will be handled?
- How to email the instructor?
- How grading works?

Programming Languages Paradigms

Programming Paradigms

- What: A style or approach to programming that provides **a framework for building a software system**. It is a **set of principles, concepts, and practices** that define the way of thinking and organizing the code to solve a specific problem.
- Main paradigms: Imperative, procedural, reactive, declarative, object-oriented, functional
- Demo

- **Imperative Programming:** This paradigm focuses on the sequence of **statements that modify the state of the program**, and how to control that sequence using conditional statements, loops, and other control flow constructs.
- **Object-Oriented Programming (OOP):** This paradigm is based on the idea of **organizing software systems as a collection of objects** that interact with each other to solve a problem.
- **Functional Programming:** This paradigm emphasizes **the use of functions** to solve problems and avoid changing the state of the program.
- **Declarative Programming:** This paradigm focuses on **describing the problem** to be solved, rather than how to solve it.

Declarative programming languages

- In declarative programming, the program **describes the desired result, rather than specifying how to achieve it**. This is in contrast to procedural programming, where the program specifies a series of steps to accomplish a task.
- One example of declarative programming is SQL (Structured Query Language), which is used to query relational databases. In SQL, a programmer specifies the criteria for selecting records from a database, and the database management system determines how to retrieve the data.
- Another example is HTML. Programmers specify web structure and content using tags, and the web browser determines how to render the page based on those tags.

Declarative programming in SQL

```
-- Define a table of students
```

```
CREATE TABLE students (  
  id INT,  
  name VARCHAR(255),  
  major VARCHAR(255),  
  gpa FLOAT  
);
```

```
-- Select all students with a GPA greater than 3.0
```

```
SELECT id, name, major  
FROM students  
WHERE gpa > 3.0;
```


Declarative programming in HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Web Page</title>
  </head>
  <body>
    <h1>Welcome to My Web Page</h1>
    <p>This is a paragraph of text.</p>
    <ul>
      <li>Item 1</li>
      <li>Item 2</li>
      <li>Item 3</li>
    </ul>
  </body>
</html>
```


Functional Programming

1

Based on recursive definitions.

- They are inspired by a computational model called **lambda calculus**, developed by Alonzo Church in the 1930s.

2

A program is viewed as a mathematical function that transforms an input to an output. It is often defined in terms of simpler functions.

- We will see many examples of functional programming in multiple languages (e.g., Java, Python, OCaml).

Functional Programming in Python

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

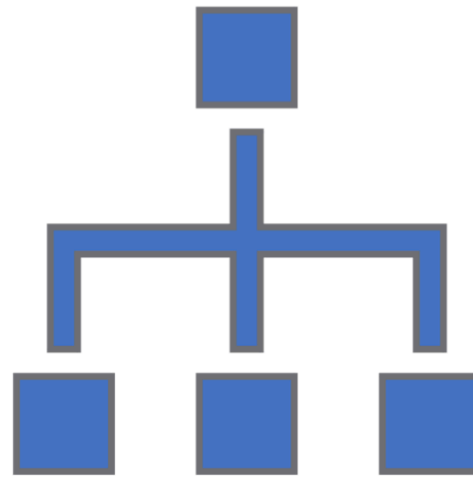
even_numbers = filter(lambda x: x % 2 == 0, numbers)
squared_numbers = map(lambda x: x ** 2, even_numbers)
sum_of_squares = sum(squared_numbers)
```

Functional Programming in Java

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);  
  
int sumOfSquares = numbers.stream()  
    .filter(x -> x % 2 == 0)  
    .map(x -> x * x)  
    .reduce(0, Integer::sum);
```

Functional Programming in Ocaml

```
let numbers = [1; 2; 3; 4; 5; 6; 7; 8; 9; 10];;  
  
let is_even x = x mod 2 = 0;;  
  
let sum_of_squares =  
  List.filter is_even numbers  
  |> List.map (fun x -> x * x)  
  |> List.fold_left (+) 0;;
```



- One more example. Three paradigms.
- Implementing the greatest common divisor (GCD) solution in different paradigms and different languages.

The GCD problem: pseudocode

```
function gcd(a, b)
  while a  $\neq$  b
    if a > b
      a := a - b;
    else
      b := b - a;
  return a;
```

Paradigm 1:

Imperative Programming in Python

```
1 def gcd (x, y):  
2     while (x!=y):  
3         if (x>y): x = x-y  
4         else: y = y -x  
5     return x  
6  
7 x = 15  
8 y = 40  
9 print (gcd(x,y))
```


Paradigm 2:

Object-Oriented Programming in Java

/MyClass.java

```
1 public class MyClass {
2
3
4
5     static int gcd (int x, int y){
6         while (x!=y){
7             if (x>y) x = x-y;
8             else y = y -x;
9         }
10        return x;
11    }
12
13
14
15
16    public static void main(String args[]) {
17        int x=10;
18        int y=25;
19
20
21        System.out.println("GCD of x+y = "+ gcd(x,y));
22    }
23 }
```


Paradigm 3:

Functional Programming in Ocaml

```
1 let rec gcd (x, y) = if x > y then gcd (x-y, y) else if x < y then gcd (x, y -x) else x ;;
2
3 let r= gcd(15, 40);;
4
5 print_int r;;
6 |
```

Why Harvard, MIT, Stanford, Cambridge all teach functional programming



<https://youtu.be/6APBx0WsgeQ>

Summary

- Imperative, functional, object-oriented paradigms
- A significant part of understanding programming is about understanding programming language paradigms
- We will focus on functional programming soon